



JXPAMG: a parallel algebraic multigrid solver for extreme-scale numerical simulations

Xiaowen Xu^{1,4} · Xiaoqiang Yue² · Runzhang Mao³ · Yuntong Deng³ · Silu Huang¹ · Haifeng Zou³ · Xiao Liu³ · Shaoliang Hu^{1,4} · Chunsheng Feng² · Shi Shu² · Zeyao Mo^{1,4}

Received: 8 January 2022 / Accepted: 5 September 2022 / Published online: 25 October 2022
© China Computer Federation (CCF) 2022

Abstract

JXPAMG is a parallel algebraic multigrid (AMG) solver for solving the extreme-scale, sparse linear systems on modern supercomputers. JXPAMG features the following characteristics: 1) It integrates some application-driven parallel AMG algorithms, including α Setup-AMG (adaptive Setup based AMG), AI-AMG (algebraic interface based AMG) and AMG-PCTL (physical-variable based coarsening two-level AMG); 2) A hierarchical parallel sparse matrix data structure, labeled hierarchical parallel Compressed Sparse Row (hpCSR), that matches the computer architecture is designed, and the highly scalable components based on hpCSR are implemented; 3) A flexible software architecture is designed to separate algorithm development from implementation. These characteristics allow JXPAMG to use different AMG strategies for different application features and architecture features, and thereby JXPAMG becomes aware of changes in these features. This paper introduces the algorithms, implementation techniques and applications of JXPAMG. Numerical experiments for typical real applications are given to illustrate the strong and weak parallel scaling properties of JXPAMG.

Keywords Algebraic multigrid (AMG) · Parallel computing · Sparse linear solver · Preconditioner · Extreme-scale computing

This work is supported by the Science Challenge Project (TZT2019) and NSFC (62032023, 11971414).

✉ Xiaowen Xu
xwxu@iapcm.ac.cn
Xiaoqiang Yue
yuexq@xtu.edu.cn
Shi Shu
shushi@xtu.edu.cn
Zeyao Mo
zeyao_mo@iapcm.ac.cn

- ¹ Laboratory of Computational Physics, Institute of Applied Physics and Computational Mathematics, Beijing 100094, China
- ² National Center for Applied Mathematics in Hunan, Key Laboratory of Intelligent Computing & Information Processing of Ministry of Education, Hunan Key Laboratory for Computation and Simulation in Science and Engineering, Xiangtan University, Xiangtan 411105, China
- ³ Graduate School of China Academy of Engineering Physics, Beijing, China
- ⁴ CAEP Software Center for High Performance Numerical Simulation, Beijing, China

1 Introduction

Algebraic multigrid (AMG) algorithms (Brandt et al. 1982; Ruge and Stüben 1987) are one of the most frequently used algorithms to solve large-scale sparse linear systems arising from the solution of partial differential equations (PDEs). Over the past 40 years, AMG solvers have been widely used in scientific, engineering and industrial applications such as structural mechanics (Ruge 1986; McCormick and Ruge 1989; Adams et al. 2003, 2004; Brezina et al. 2006; Tian et al. 2019), radiation hydrodynamics (Baldwin et al. 1999; Henson 2003; Falgout et al. 2005; Yue et al. 2015; Xu and Mo 2017), semiconductor devices (Füllenbach et al. 2000; Clees and Stüben 2002), oil and gas exploration (Clees and Ganzer 2010; Gries et al. 2014), groundwater flow simulations (Stüben et al. 2003) and the automotive industry (Stüben 2001; Stüben et al. 2017) as the key components for large-scale computing in these applications. Since AMG algorithms play an important role in practical applications, AMG solvers are used as the benchmark code for performance assessment of exascale computers (CORAL-2 Benchmarks 2017).

During the past two decades, the academic and applied sectors have developed a variety of AMG solvers, e.g., BoomerAMG from LLNL (Henson and Yang 2002), MueLu from SNL (MueLu Web page 2019), LAMG (Lean Algebraic Multigrid) from LANL (Joubert and Cullum 2006), GAMG (Geometric Algebraic Multigrid) from ANL (PETSc Web page 2022), SAMG (Algebraic Multigrid methods for Systems) from SCAI (SAMG Web page 2022) and AmgX from NVIDIA (Sellappan and Desam 2015), which have achieved great success in different application scenarios. The existing AMG solvers mentioned above are designed for specific applications or machine architectures by developing different types of AMG algorithms, and there does not exist a general AMG solver that can be used in all or most scenarios. For instance, as the first parallel AMG solver in the world, BoomerAMG in the hypre library (HYPRE Web page 2022) mainly focuses on the classical AMG. As the core component of Trilinos packages (Trilinos Web page 2020), MueLu mainly focuses on smoothed aggregation AMG (SA-AMG) (Vanek et al. 1996, 2001) and is mainly used in structural mechanics. LAMG is a non-public AMG solver. GAMG in PETSc (PETSc Web page 2022) also mainly focus on SA-AMG. As the first commercial AMG software, SAMG is designed for industrial applications such as automotive designing. The AmgX, which is developed by ANSYS and NVIDIA, is specially designed for GPU architectures. In recent years, to adapt to the new architectures of exascale computers, the R&D focus of the solvers such as BoomerAMG, MueLu and GAMG have switched to the reconstruction and performance optimization on GPU architectures (Heroux et al. 2019). Developing efficient parallel AMG solvers for complex applications is an important issue in the extreme-scale numerical simulations.

AMG efficiency depends on both application features and architecture features. On one hand, the complex features such as multi-physics couplings, multi-material, non-linearity, and strong discontinuity, could deteriorate the convergence rate of AMG. On the other hand, the complex architecture of supercomputer, e.g., massive, heterogeneous concurrency, could result in bad parallel efficiency of AMG.

The JXPAMG (Parallel AMG Solver Developed by JiuSuo and XTU) is a parallel AMG solver jointly developed by IAPCM (JiuSuo) and XTU as a feature-aware AMG solver with high parallel scalability for extreme-scale computing. Based on classical AMG algorithms (Brandt et al. 1982; Ruge and Stüben 1987) and Krylov subspace iterative methods (Saad 2003), JXPAMG integrates various application-driven parallel AMG algorithms (Yue et al. 2015, 2021; Xu and Mo 2017; Xu et al. 2016, 2020, 2009; Zhou et al. 2012) developed for practical applications, and features a hierarchical parallel sparse matrix data structure that matches the modern computer architecture as well as highly scalable components and operations (including parallel smoothers,

RAP and SpMV). In addition, JXPAMG also features a flexible software architecture that separates algorithms, components (e.g., coarsening, transfer operators and smoothing) and operations (e.g., SpMV, RAP, etc.) from one another. These characteristics allow JXPAMG to use different AMG algorithms and implementation strategies for different application features and supercomputer architecture features, and thereby become aware of changes in these features. Currently, by seamlessly integrating in parallel programming frameworks JASMIN (Mo et al. 2010) and JAUMIN (Liu et al. 2019), JXPAMG are being used in various applications, such as laser fusion, engineering mechanics, electromagnetic environments, and combustion simulations, and enables these applications to efficiently solve sparse linear systems with hundreds of millions to ten billions of degrees of freedom (DOFs) on tens of thousands to hundreds of thousands of China's domestic supercomputers.

This paper is organized as follows. Section 2 introduces the basic principles of AMG algorithms as well as their efficiency issues in practical applications and design ideas of the JXPAMG solver. Section 3 introduces algorithms, data structure, software architecture and user interfaces of JXPAMG. Section 4 shows parallel performance and some typical simulation results of JXPAMG. Section 5 summarizes the conclusions.

2 AMG efficiency issues

2.1 AMG basics

We consider the sparse linear system $Ax = b$, where $A \in R^{n \times n}$, $x \in R^n$ and $b \in R^n$ are the sparse matrix, unknown vector, and right-hand-side (RHS) vector respectively. Since the AMG algorithm does not rely on grid information, we usually consider the adjacency graph $G(\Omega, E)$ of the sparse matrix as a virtual grid, where $\Omega = \{x_1, x_2, \dots, x_n\}$ is the node set representing unknown variables, and E is the edge set representing the relations between unknown variables. For simplicity, Ω is usually denoted as a grid.

Taking a two-level case as an example, the classical AMG procedure (Ruge and Stüben 1987) is described in *Algorithm 1*.

Algorithm 1 Setup and Solve Phases of AMG (Two-Level Case)

1. *Setup Phase*: Coarse level construction
 - 1.1 *Coarsening*: selecting a subset $\Omega^c \subseteq \Omega$ as coarse level;
 - 1.2 Constructing *Prolongation* and *Restriction* Operators;

$$P : \Omega^c \rightarrow \Omega, R : \Omega \rightarrow \Omega^c;$$

1.3 Constructing Coarse Operators: $A^c = RAP$ (Galerkin Method);

2. Solve Phase: performs the following cycle process iteratively until it meets the convergence criteria.

2.1 Pre-smoothing : $x \leftarrow S^{\mu_1}(A, b, x)$;

2.2 Computing residual and restriction : $b^c = R(b - Ax)$;

2.3 Solving on coarser level : $A^c x^c = b^c$;

2.4 Prolongation and correction : $x = x + Px^c$;

2.5 Post-smoothing: $x \leftarrow S^{\mu_2}(A, b, x)$.

As described in **Algorithm 1**, AMG has two phases: Setup and Solve. The Setup phase constructs AMG components, including coarse levels, transfer operators, and coarse-level operators, thereby a purely algebraic grid hierarchy is constructed. Based on the grid hierarchy, the Solve phase performs the standard multigrid cycles, e.g., V-Cycle and W-Cycle. For the detailed AMG algorithms, we refer to relevant references.

2.2 Efficiency issues

In practical application scenarios, AMG algorithms are usually used as the preconditioners to improve the convergence rates of the Krylov subspace iterative methods (Saad 2003). The AMG efficiency mainly depends on two factors: the number of iterations and the parallel scalability of a single iteration. The former depends on how the AMG components match the application features, while the latter depends on how the parallel implementations of AMG match the architecture features of supercomputers. The basic principles of a multigrid algorithm are based on the interplay of smoother and coarse grid correction which complement each other. For AMG, the plain smoothers, e.g., Jacobi or Gauss–Seidel methods, are usually used. Hence, The AMG efficiency largely depends on whether the coarse levels constructed in Setup phase matches the application features and architecture features.

In classical AMG algorithm, the coarse level matrix is usually constructed using the Galerkin method which performs a tri-matrix multiplication denoted by RAP (step 1.3 in **Algorithm 1**). Since RAP cannot maintain the sparsity pattern of coarse matrix as the number of levels increases, the coarse matrices become increasingly dense, causing the storage and the communication overhead to grow, which leads to a performance bottleneck. AMG uses the following operator complexity (C_{op}) to describe the overhead introduced by the coarse grid matrix:

$$C_{op} = \frac{\sum_l |A^l|}{|A^0|}$$

where A^l denotes the matrix of mesh l , and $|A^l|$ denotes the number of nonzero entries in A^l . $A^0 = A$ is the matrix of the finest level ($l = 0$). C_{op} reflects the number of additional nonzero entries introduced by the AMG iteration matrix (which is usually not computed explicitly) compared with the finest level matrix, and can be used as an important indicator of the computational cost of each iteration and storage requirements.

Previous studies, e.g., in Sterck et al. (2006), show that high operator complexity is not suitable for large-scale parallel computing owing to expensive memory requirements and communication overheads, and low operator complexity is essential. However, in practical applications, AMG algorithms with low operator complexity are still faced with the following two challenges (Xu 2019):

- Complex features in practical applications such as multiphysics couplings, multi-material, strong nonlinearity and strong discontinuity could deteriorate the convergence rate of the AMG algorithm and increase the number of iterations drastically.
- The complex architecture features of a supercomputer such as hierarchical, massive and heterogeneous concurrency could affect the parallel efficiency of AMG algorithm.

Hence, developing the AMG solvers with both fast convergence and high parallel scalability is essential for the large-scale computations. On one hand, we need to develop featured-based components based on application features. On the other hand, we need to develop highly scalable parallel implementation techniques to adapt to the computer architecture features. In addition, we also need to design a software architecture that can quickly integrate feature-based algorithms and parallel implementations and provide AMG solvers with the feature-aware adaptive tuning capability. These are exactly the goals of JXPAMG described in this paper.

3 JXPAMG: algorithms, implementation, and architecture

3.1 Application-driven AMG algorithms

In addition to classical AMG algorithms and Krylov subspace iterative methods, JXPAMG integrates various feature-based AMG algorithms (Xu and Mo 2017; Xu et al. 2016, 2020, 2009; Zhou et al. 2012; Yue et al. 2021) for

practical application features. Main algorithms integrated by JXPAMG are described briefly below, and their components and parameters are listed in Table 1 in Sect. 3.4.

- **C-AMG: Classical AMG algorithms** (Brandt et al. 1982; Ruge and Stüben 1987). Please refer to **Algorithm 1** in Sect. 2.1 for details. JXPAMG integrates typical components such as smoother, coarsening, interpolator and coarse matrix construction.
- **AI-AMG: Algebraic interface based AMG algorithms** (Xu and Mo 2017). These algorithms are designed to solve multi-scale matrices. The so-called “multi-scale” refers to a scenario where the values of the off-diagonal entries in the same row of the matrix differ significantly (by a few or a dozen orders of magnitude). Compared with a single-scale matrix, the AMG efficiency would decrease for multi-scale matrices. Xu and Mo (2017) introduced the concept of an algebraic interface to describe the key features of a multi-scale matrix, and designed an algebraic interface based AMG algorithm. Please refer to Xu and Mo (2017) for details. These algorithms are suitable for solving such features as multi-physics couplings, multi-material and discontinuous coefficients.
- **α Setup-AMG: Adaptive Setup based AMG algorithms** (Xu et al. 2016, 2020). These algorithms are designed to solve sequences of linear systems. Their main idea is to apply different coarsening strategies to different systems in the sequence by designing Setup conditions, thus minimizing the AMG algorithm’s Setup overhead and improving their parallel scalability. These methods are suitable for solving linear systems derived from time dependent nonlinear partial differential equations. The components in C-AMG or AI-AMG can be also selected for α Setup-AMG.
- **AMG- α PCTL: Adaptive physical-variable based coarsening two-level AMG algorithms** (Xu et al. 2009; Zhou et al. 2012). These algorithms are designed to solve three-temperature energy equations in radiation hydrodynamics (Xu et al. 2009). Their main idea is to leverage the special structure of a three-temperature coupled system and use specially designed coarsening strategies

to split the coupled system into 4 subsystems that can be efficiently solved by AMG.

- **Single-level algorithms:** All the smoothers and coarsest solvers can degrade to single-level algorithms, including (Blocked) Jacobi, Gauss-Seidel, SOR, SSOR, ILU and GE.

In JXPAMG, Krylov subspace iterative methods such as CG, BiCGSTAB, GMRES(m), FGMRES(m) and COGMRES are integrated.

3.2 hpCSR: hierarchical parallel sparse matrix data structure

Data structure of sparse matrix is the basis of an AMG solver. To achieve high efficiency, the data structure should match the supercomputer architecture. The modern supercomputer systems usually feature a “Node-Processor-Core” hierarchical architecture. In other words, a computer system consists of many Nodes interconnected through a high-speed network; each Node consists of multiple Processors; and each Processor consists of Multi-Cores/Many-Cores. Processors include general-purpose processors (CPU) and various accelerators (e.g., GPU, MIC, DCU and Matrix3000). For many current supercomputers, Node feature a mixed architecture with CPU and accelerator.

To match this “Node-Processor-Core” three-level architecture, a hierarchical parallel CSR (hpCSR) data structure based on the traditional parallel CSR data structure (ParCSR) was designed in JXPAMG. Concretely, if the processes in the ParCSR are mapped to Cores, it shall be extended by two levels, and therefore the hpCSR shall be a three-level data structure corresponding to the “Node-CPU-Core” hierarchy; if the processes in the ParCSR are mapped to CPUs, it shall be extended by one level, and therefore the hpCSR shall be a two-level data structure corresponding to the “Node-CPU” hierarchy; if the processes in the ParCSR are mapped to Nodes, the hpCSR shall be exactly the ParCSR, which is a single-level data structure.

Table 1 Main components of AMG in JXPAMG

Components	Options	Default
Smoother	w-Jacobi, Gauss–Seidel, w-SOR, w-SSOR, sssHybrid (Block Jacobi), ILU	Hybrid
Coarsening	RS, RS0, CLJP, Falgout, PMIS, HMIS, Relaxed-type, AI-type, aggressive strategy	PMIS
Interpolator	Classical, direct, standard, one-point, multi-pass, extended, truncated strategy	Standard
Coarser operator	Galerkin, non-Galerkin	Galerkin
Coarsest solver	Gaussian Elimination (GE), iterative methods	GE

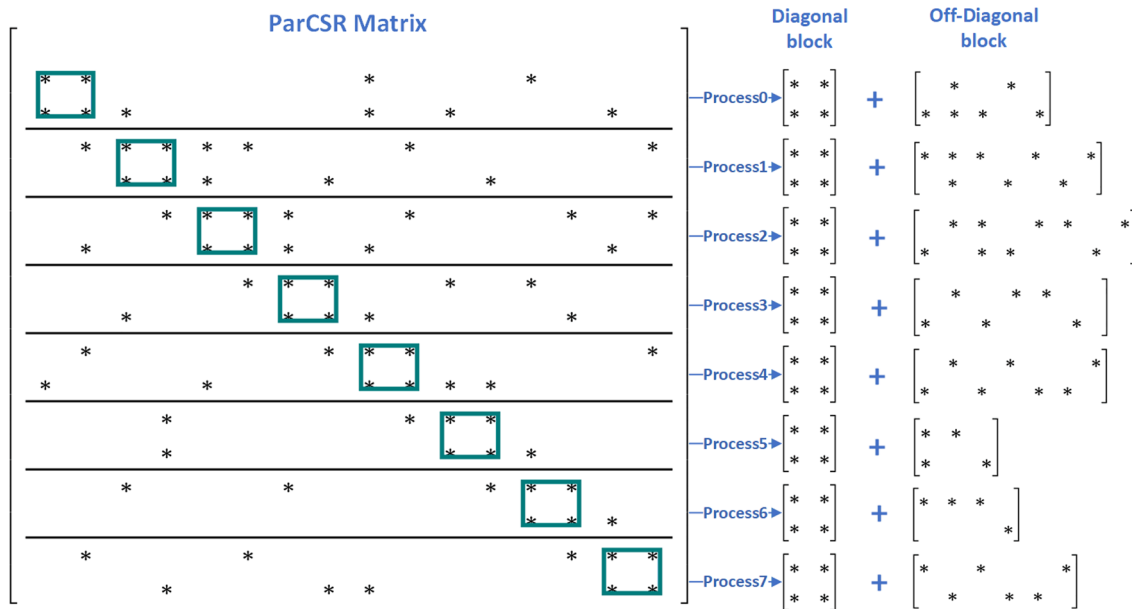


Fig. 1 ParCSR Matrix

Let us take a matrix of size 16 as an example, Figure 1 illustrates the ParCSR matrix data structure.

As shown in Fig. 1, the ParCSR matrix is partitioned into 8 blocks by row, and each block is distributed to a process, which consists of a diagonal block and an off-diagonal block.

Let processes be mapped to Cores. Suppose a computer system consists of two Nodes with each Node consisting of two CPUs and each CPU consisting of two Cores, then the corresponding hpCSR matrix is illustrated in Fig. 2.

As shown in Fig. 2, compare with ParCSR format, the hpCSR need to extend the diagonal blocks of CPU level and

Node level. Let us take the diagonal blocks of CPU level as an example to describe how they are created. Since one CPU consists of two Cores, each diagonal block of CPU level (red boxes) are created by extending two diagonal blocks of Core level (blue boxes). Hence, there are 4 diagonal blocks of CPU level. To create a diagonal block of CPU level, each process needs to obtain the partitioning of CPU level via the information of the diagonal block of the other process in the same CPU, and add new nonzero entries (red entries in Fig. 2) which are stored as a CSR format. Using the similar

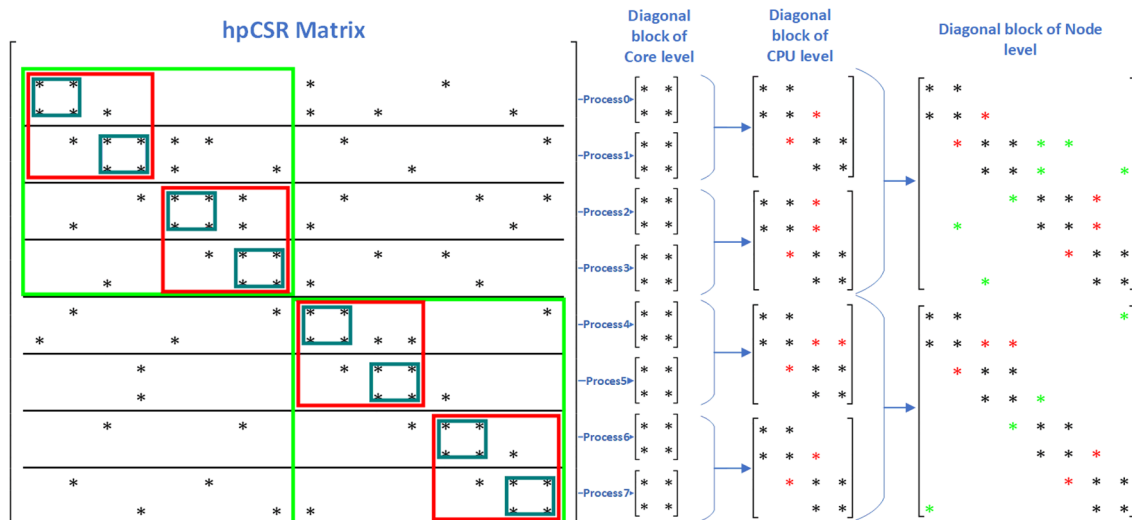


Fig. 2 hpCSR Matrix

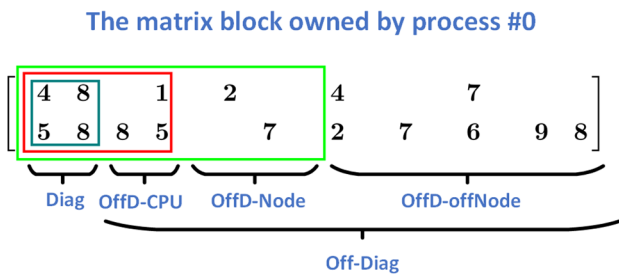


Fig. 3 The matrix block owned by process #0 and its division

way, diagonal blocks of Node level (green boxes) can be also created.

Figure 3 gives the matrix block of process #0 as an example to introduce the implementation of hpCSR, which also contains the diagonal and off-diagonal blocks of ParCSR to ensure that the original interface based ParCSR works fine in JXPAMG. Moreover, hpCSR stores the diagonal blocks of CPU and Node levels from the off-diagonal blocks and numerous hardware-aware information, such as the number of cores in each node, the starting of the process ID in the same CPU, etc. For more details see Fig. 4.

Fig. 4 hpCSR data structure

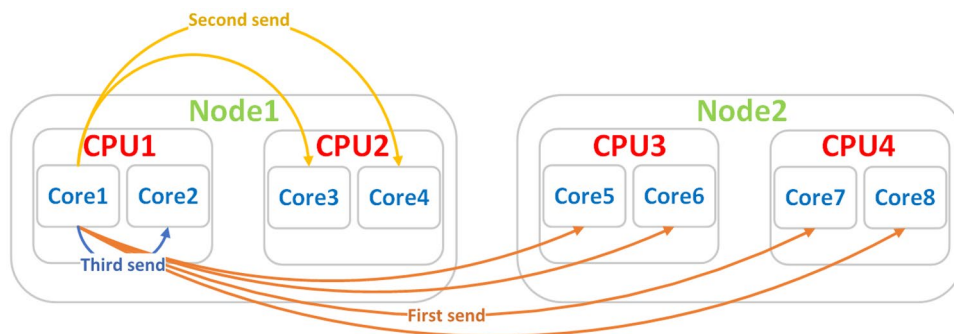
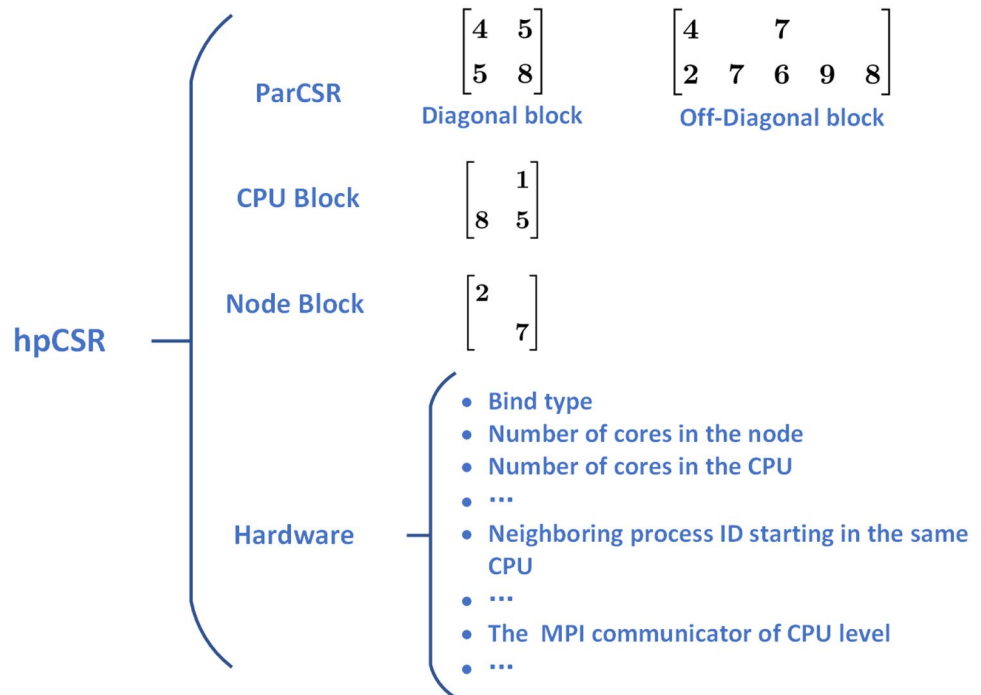


Fig. 5 Taking the Beast communication of the Process on Core1 as an example to illustrate the hierarchical communication scheduling strategy, which includes three steps: firstly, the Process sends messages to those Processes of Node2 (in orange lines); secondly, the

Process sends messages to those Processes of CPU2 of the same Node (in yellow lines); finally, the Process sends messages to other Processes within the same CPU (in blue lines)

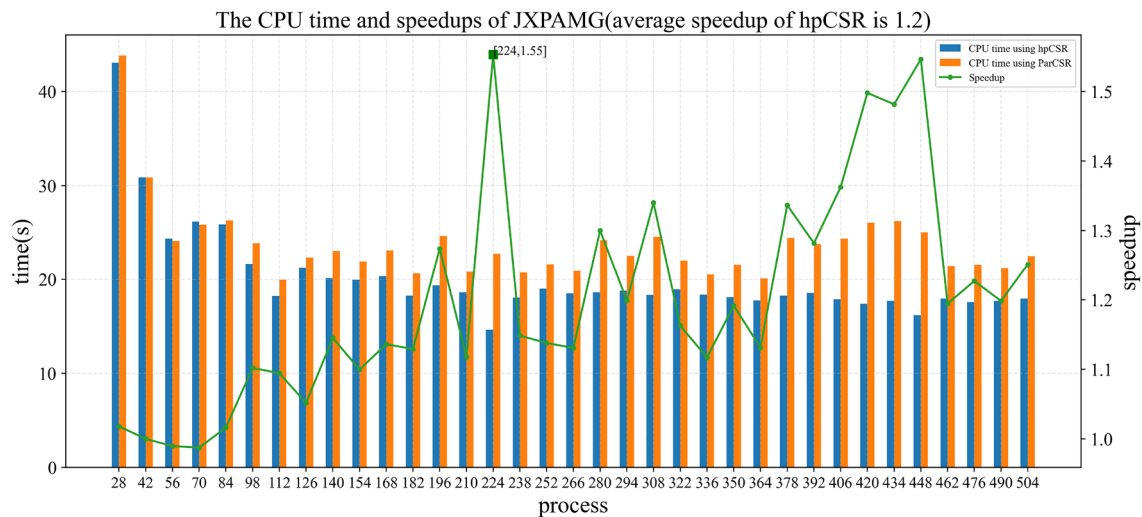


Fig. 6 The CPU time and speedups of JXPAMG in hpCSR and ParCSR when solving a discretized 3D diffusion equation with different numbers of processes. The orange and blue bars represent the

CPU time measured in seconds for hpCSR and ParCSR, respectively while the green line shows the speedups of hpCSR relative to ParCSR

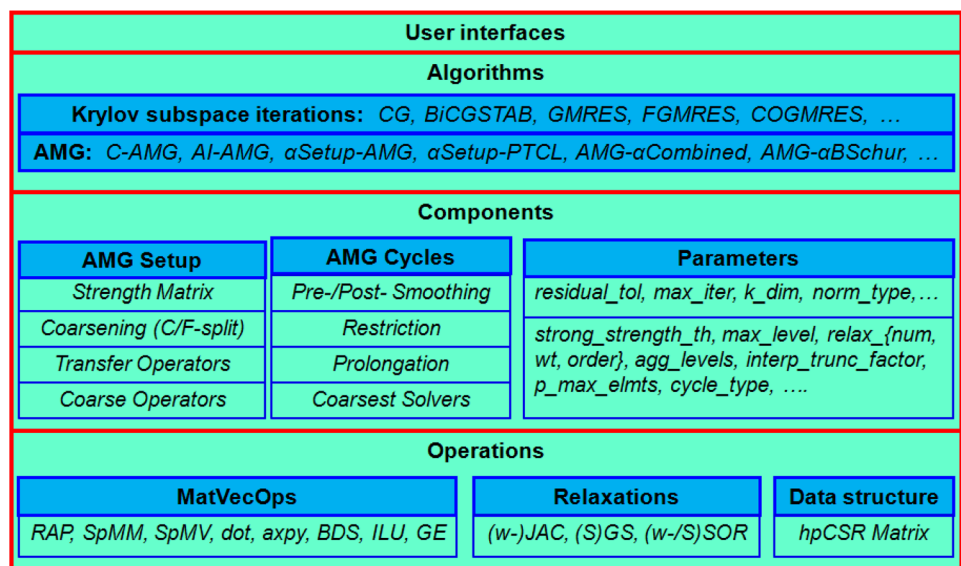
Being aware of the architecture features, the hpCSR can leverage them to better perform basic operations. For example, as shown in Fig. 5, the hpCSR can distinguish communications among intra-CPU, intra-Node and inter-Node, thus allowing us to reduce communication cost of AMG components such as RAP and SpMV operations in a hierarchical communication scheduling strategies. Figure 6 shows the performance improvement of hpCSR when solving a 3D diffusion equation discretized by the finite volume scheme with the mesh size $128 \times 128 \times 128$, i.e., 2,097,152 DOFs. Compared with ParCSR, the average speedup of hpCSR is 1.2. In addition, exploiting the multi-level blocks of different

granularities in hpCSR, the specific architecture-aware parallel smoothers are also designed in JXPAMG.

3.3 Software architecture

JXPAMG adopts a reuse-based software engineering strategy. As shown in Fig. 7, its software architecture consists of the user interfaces layer, algorithms layer, components layer and operations layer. The user interfaces layer incorporates relevant interfaces used by the user to call AMG algorithms and settings (refer to the next section for its usage). The algorithms layer consists of general Krylov subspace iterative methods, AMG algorithms and their combinations.

Fig. 7 Software architecture of JXPAMG



The components layer integrates basic components of AMG algorithms and related parameters, including coarsening, transfer operators, coarse grid matrix construction, smoother, and coarse level solver, etc. The operations layer integrates the matrices & vectors data structure and their relevant operations required by the algorithms layer and components layer.

With the hierarchical software architecture, JXPAMG separates algorithm integration from implementation. On one hand, it supports quick integration of various algorithms and components. On the other hand, the operations layer can be optimized for performance independent from the algorithms and components layer.

3.4 User interface

To solve the linear system, the user only needs to provide the matrix and RHS vector using the user interface of JXPAMG. The matrix provided by the user can be a ParCSR matrix which will be converted into an hpCSR matrix by JXPAMG. The user is allowed to select all the algorithms and their combinations listed in the algorithms layer as shown in Fig. 7. The main components of these algorithms are listed in Table 1. Please refer to user manual for details on other components and parameters.

We take AMG_GMRES as an example to illustrate the usage of the user interface. The procedure consists of 7 steps as follows.

```
//1.Create AMG Solver
JX_PAMGCreate(&amg_solver);

//2.Set Parameters for AMG
JX_PAMGSetMaxLevels(amg_solver, max_levels);
JX_PAMGSetMaxIter(amg_solver, 1);
JX_PAMGSetCycleType(amg_solver, cycle_type);

//3.Create GMRES Solver
JX_ParCSRGMRESCreate(comm, &solver);

//4.Set Parameters for GMRES
JX_GMRESSetMaxIter(solver, max_iter);
JX_GMRESSetTol(solver, tol);

//5.Set AMG Preconditioner
JX_GMRESSetPrecond(solver, (JX_PtrToSolverFcn)JX_PAMGPrecond,
                   (JX_PtrToSolverFcn)JX_PAMGSetup, amg_solver);

//6. Setup Phase
JX_PAMGSetup(amg_solver, (JX_hpCSRMatrix)matrix);
JX_GMRESSetup(solver, (JX_Matrix)matrix,
              (JX_Vector)rhs, (JX_Vector)sol);

//7. Solve Phase
JX_GMRESSolve(solver, (JX_Matrix)matrix, // preOperator
              (JX_Matrix)matrix, (JX_Vector)rhs, (JX_Vector)sol);
```

Table 2 Weak scalability results of JXPAMG, 3D Poisson equation

#Nodes	#Cores	Threads per process	DOFs	C_{op}	#Iterations	Time(s)	Parallel Efficiency (per iteration)
1	32	1	786,432	2.46	27	0.5	–
8	256	1	6,291,456	2.47	34	0.9	–
64	2048	1	50,331,648	2.48	47	2.7	100%
512	16,384	2	402,653,184	2.48	65	3.9	95.0%
1024	32,768	2	805,306,368	2.36	77	4.3	72.1%
2048	65,536	4	1,610,612,736	2.35	91	7.5	69.5%
2100	67,200	4	1,651,507,200	2.36	90	7.9	65.5%

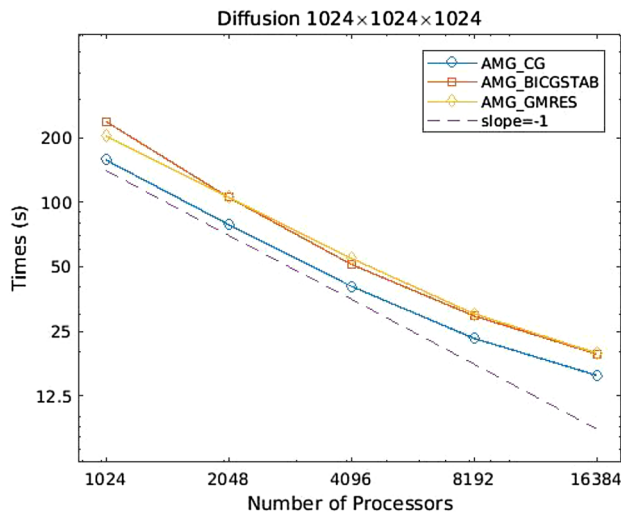


Fig. 8 Strong scalability of JXPAMG on Tianhe Supercomputer

Remark JX_Matrix, JX_Vector and JX_hpCSRMatrix in step 6 and step 7 are three abstract structure pointers for code reuse and maintenance.

4 Parallel performance and applications

In this section, we show the performance and applications of JXPAMG with some typical problems. Firstly, we test the parallel performance of JXPAMG using a model problem. Then, we show the simulation results of JXPAMG using two typical practical problems and show its efficiency.

4.1 Parallel performance of JXPAMG

In this subsection, we use the 3D Poisson equation $-\Delta u = f$ to test the parallel scalability of JXPAMG. 7-point finite difference method on a uniform mesh is used for discretization. The AMG preconditioned Conjugate Gradient (AMG-CG) method is used to solve the resulting linear systems. For AMG, the default options of components listed in Table 1 are used, and the strong connection threshold is set to 0.5. Except for special statement, the residual tolerance is chosen to be 10^{-10} , i.e., reducing the 2-norm of the residual by 10 orders of magnitude.

Weak scalability tests were performed on a petaflops supercomputer with a fixed mesh size per Core of $32 \times 32 \times 24$, and the number of Cores increasing from 32 to 67,200. Test results are given in Table 2.

The results in Table 2 show that JXPAMG has approximately optimal computational complexity and high parallel scalability. First, the operator complexity is low and remains constant as the Cores and DOFs increase. The iteration

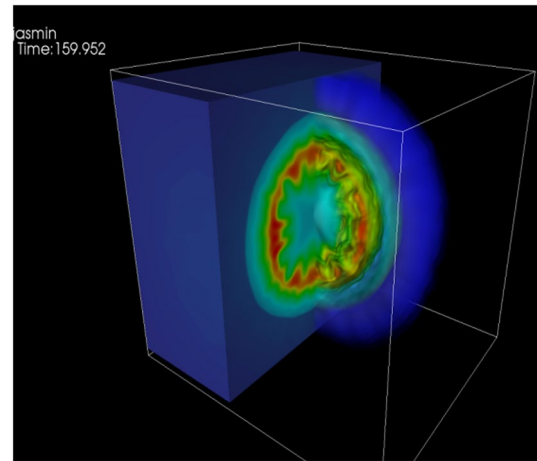


Fig. 9 A typical 3D simulation results of hydrodynamic instabilities during the processes of ICF capsule implosion with 21.4 billion DOFs on 32,000 CPU cores

numbers increase approximately linearly with the DOFs. Secondly, the parallel efficiency of a single iteration reaches 65.5% as the Core number increases from 2048 to 67,200.

Figure 8 shows the strong scalability of JXPAMG on the Tianhe Supercomputer with each node containing one FT CPU and 16 GB of memory, each CPU containing 64 cores. The fixed mesh size is $1024 \times 1024 \times 1024$, and the number of CPU core increases from 1024 to 16,384. We test three Krylov subspace iterative methods (CG, BiCGSTAB, and GMRES) with AMG as preconditioner. Again, the default options of AMG components listed in Table 1 are used, and the dimension of Krylov subspace for GMRES was set to 20. Figure 8 shows that all the three methods have achieved good parallel scalability.

4.2 Typical application-I: Radiation hydrodynamic simulation

The radiation hydrodynamics (RHD) computations play a crucial role in simulations of problems in fields such as astrophysics and inertial confinement fusion (ICF) (Pei 2007). RHD is a multi-physics process containing both hydrodynamic motion and energy transport. In practical simulations, the need for large time step size and stability dictates the use of an implicit scheme for RHD equations, which means that linear solvers are in great demand for RHD computation. However, complicated application features such as multiphysics coupling, multimaterial and strong nonlinearity make it very difficult to solve these equations, which is a well known challenging problem (Pei 2007). In fact, RHD computation is always being an important application scenario of AMG algorithms (Baldwin et al. 1999; Henson 2003; Falgout et al. 2005; Yue et al. 2015;

Table 3 Performance results for Sanxia Dam structure analysis

Refine	Mesh size	Dofs	#Cores	#Iterations	CPU Time(s)
0	0.32 billion	0.16 billion	512	243	368.6
1	2.56 billion	1.30 billion	2048	343	466.7
2	20.51 billion	10.34 billion	32,768	581	1261.1

Xu and Mo 2017), and also the most application scenario of JXPAMG.

In the past two decades, a variety of AMG algorithms for different RHD models were proposed (Baldwin et al. 1999; Henson 2003; Falgout et al. 2005; Yue et al. 2015, 2021, 2017; Xu and Mo 2017; Xu et al. 2016, 2020, 2009; Zhou et al. 2012; Xiao et al. 2003). JXPAMG integrates various RHD-driven AMG algorithms, including AMG- α Combined (Yue et al. 2015) and AI-AMG (Xu and Mo 2017) for multi-scale matrices, AMG- α PCTL (Xu et al. 2009; Zhou et al. 2012) for three-temperature models, and AMG- α BSchur (Yue et al. 2021) for multi-group models, and α Setup-AMG (Xu et al. 2016, 2020) for sequences of systems arising from long time-integration process. Please refer to relevant references for more details on these algorithms.

We show a typical example of an instability simulation of the three-dimensional hydrodynamics interface of an ICF capsule implosion process. In this example, a multigroup diffusion model with 20 groups was used to simulate the instability development of the hydrodynamics interface during the stagnation phase of the implosion process. The mesh size is $1024 \times 1024 \times 1024$ (about 1.07 billion), since the energy equations consisting of 20 groups lead to 21.4 billion DOFs. To solve this problem, the RHD code ran for 0.1 ns of physical time, with the overall simulation requiring 53,470 time steps. JXPAMG used AI-AMG and α Setup-AMG algorithms, the total CPU time is 7 days on 32,000 CPU cores. This is the most efficient result at present. JXPAMG outperforms others open-source AMG solvers. Figure 9 shows the simulation results.

Currently, for 2D RHD models, AMG algorithms in JXPAMG are relatively mature. However, for 3D multigroup models, as the grid resolution increases, efficiencies of the existing AMG algorithms will not meet simulation demands and therefore ongoing and in-depth research is required in the future.

4.3 Typical application-II: Structural mechanics analysis

Structural mechanics analyses are widely used in various applications where a multiphysical coupled system of partial differential equations need to be solved, which is also a typical challenge problem. AMG algorithms were used to solve these problems (Ruge 1986; McCormick and Ruge 1989), and were later successfully applied to the structural mechanics analysis of human bones (Adams et al. 2003, 2004) and crystallization (Brezina et al. 2006). Especially, JXPAMG was successfully applied to the structural mechanics analysis and safety assessment of dams (Tian et al. 2019). In these applications, there are thermo-mechanical-coupled contact problems needed solving. Physics-based preconditioners are used to convert the original problem into several Poisson-like equations which AMG algorithms can be used to solve the problems efficiently. For instance, JXPAMG was used to solve the linear systems with 1.1 billion DOFs on 16000 CPU cores of Tianhe-II supercomputer in the case of the structural analysis of Tianshengqiao dam. Please refer to reference (Tian et al. 2019) for more details.

Recently, JXPAMG was successfully applied to the static mechanics analysis of the Sanxia dam. The simulation used a real Sanxia dam model covering ten square kilometers of ground and dam sections, took the dam's entire gravity and hydraulic load into consideration, and applied fixed boundary conditions to the bottom of the dam. In this simulation, the coarsest mesh size is 0.32 billion tetrahedral elements, and the mesh refined twice, the finest mesh size is 20.5 billion tetrahedral elements with 10.3 billion DOFs. Figure 10 shows the simulation results.

Fig. 10 The simulation results of the stress for the Sanxia Dam, 10.3 billion DOFs, 32,768 cores

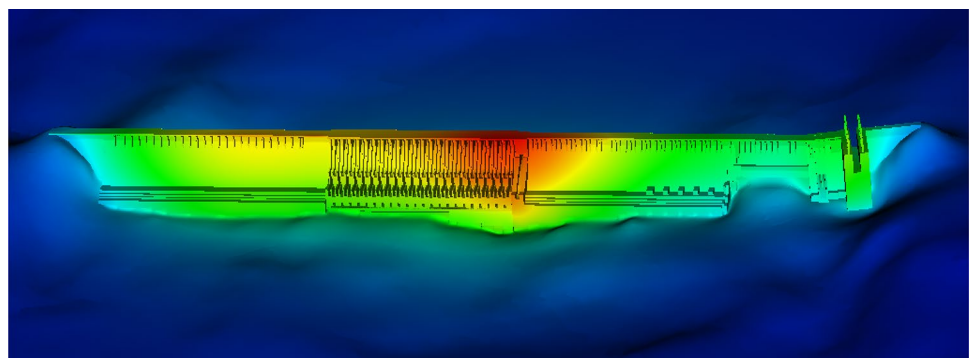


Table 3 shows the performance test results of JXPAMG for this problem. As shown in the table, JXPAMG is highly scalable and can solve the linear systems with over 10 billion DOFs on 32,768 CPU cores within 21 min. To the best of our knowledge, no existing other solvers can solve this problem. This is so far the largest scale dam static analysis, and for the first time that a static analysis of the entire Sanxia dam was performed, which provides strong support for dam behavior assessment.

5 Conclusions

This paper introduces the algorithms, parallel implementation techniques, software architecture, user interfaces as well as typical applications of the JXPAMG solver. Firstly, JXPAMG features a hierarchical parallel sparse matrix data structure hpCSR that matches the hierarchical architecture of supercomputers. Performance results show that JXPAMG can run efficiently on domestic supercomputers. Secondly, JXPAMG integrated various feature-based AMG algorithms, and is widely used in the fields of scientific and engineering computation. Especially, JXPAMG can be applied to radiation hydrodynamic simulations and structural mechanics analyses to efficiently solve linear systems with tens of billions of DOFs.

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no conflict of interest.

References

- Adams, M., Bayraktar, H., Keaveny, T., Papadopoulos, P.: Applications of algebraic multigrid to large-scale finite element analysis of whole bone micro-mechanics on the IBM SP, SC'03, 2003.
- Adams, M., Bayraktar, H., Keaveny, T., Papadopoulos, P.: Ultra scalable implicit finite element analysis in solid mechanics with over a half billion DOFs, SC'04, 2004.
- Baker, A., Falgout, R., Kolev, T., Yang, U.: Multigrid smoothers for Ultra-Parallel computing. *SIAM J. Sci. Comput.* **33**, 2864–2887 (2011)
- Baldwin, C., Brown, P., Falgout, R., Graziani, F., Jones, J.: Iterative linear solvers in 2D radiation-hydrodynamics code: methods and performance. *J. Comput. Phys.* **154**, 1–40 (1999)
- Brandt, A., McCormick, S., Ruge, J.: Algebraic Multigrid (AMG) for automatic solution with application to geodetic computations, Institute for Computational Studies, POB 1852. Fort Collins, Colorado (1982)
- Brezina, M., Tong, C., Becker, R.: Parallel algebraic multigrids for structural mechanics. *SIAM J. Sci. Comput.* **27**(5), 1534–1554 (2006)
- Cleary, A., Falgout, R., Henson, V., Jones, J.: Coarse-grid selection for parallel algebraic multigrid. *Lect. Notes Comput. Sci.* **1457**, 104–115 (1998)
- Clees, T., Stüben, K.: Algebraic multigrid for industrial semiconductor device simulation, Proceedings of the 1st International Conference on Challenges in Scientific Computing, Berlin, Germany (2002)
- Clees, T., Ganzer, L.: An efficient algebraic multigrid solver strategy for adaptive implicit methods in oil-reservoir simulation. *SPE J.* **15**(3), 670–681 (2010)
- CORAL-2 Benchmarks: <https://asc.llnl.gov/coral-2-benchmarks>.
- de Sterck, H., Yang, U., Heys, J.: Reducing complexity in parallel algebraic multigrid preconditioners. *SIAM J. Matrix Anal. Appl.* **27**(4), 1019–1039 (2006)
- Falgout, R., Schroder, J.: Non-Galerkin coarse grids for algebraic multigrid. *SIAM J. Sci. Comput.* **36**, C309–C334 (2014)
- Falgout, R., Brannick, J., Brezina, M., Manteuffel, T., McCormick, S.: New multigrid solver advances in TOPS. *J. Phys.: Conf Ser.* **16**, 456–460 (2005)
- Füllenbach, T., Stüben, K., Mijalkovic, S.: Application of an algebraic multigrid solver to process simulation problems, Proceedings of the IEEE Intern. Conference on Simulation of Semiconductor Processes and Devices, Seattle(WA), USA, 225–228 (2000)
- Gries, S., Stüben, K., Brown, G., Chen, D., Collins, D.: Preconditioning for efficiently applying algebraic multigrid in fully implicit reservoir simulations. *SPE J.* **19**(4), 726–736 (2014)
- Henson, V.: New directions for algebraic multigrid: solutions for large scale multiphysics problems, LLNL Report No. UCRL-ID-151775 (2003)
- Henson, V., Yang, U.: BoomerAMG: a parallel algebraic multigrid solver and preconditioner. *Appl. Numer. Math.* **41**, 155–177 (2002)
- Heroux, M., Carter, J., Thakur, R., Vetter, J., McInners, L., Ahrens, J., Robert, J.: ECP Software Technology Capability Assessment Report-Public V1.5, (2019)
- HYPRE Web page: <https://computing.llnl.gov/projects/hypre-scalable-linear-solvers-multigrid-methods>.
- Joubert, W., Cullum, J.: Scalable algebraic multigrid on 3500 processors, *Electron. Trans. Numer. Anal.*, 23: 105–128, 2006. Also appear at Technical Report No. LAUR03–568, LANL, 2003.
- Liu, Q., Mo, Z., Zhang, A., Yang, Z.: JAUMIN: a programming framework for large-scale numerical simulation on unstructured meshes. *CCF Trans. HPC* **1**, 35–48 (2019)
- McCormick, S., Ruge, J.: Algebraic multigrid methods applied to problems in computational structural mechanics, Technical Report 19900060129, NASA. Also available in State-of-the-art surveys on computational mechanics, 237–270, New York, American Society of Mechanical Engineers, 1989.
- Mo, Z., Xu, X.: Relaxed RS0 or CLJP coarsening strategy for parallel AMG methods. *Parallel Comput.* **33**(3), 174–185 (2007)
- Mo, Z., Zhang, A., Cao, X., Liu, Q., Xu, X., An, H., Pei, W., Zhu, S.: JASMIN: a parallel software infrastructure for scientific computing. *Front. Comput. Sci. China* **4**(4), 480–488 (2010)
- MueLu Web page: <https://trilinos.github.io/muelu.html>.
- Pei, W.: The construction of simulation algorithm for laser fusion. *Commun. Comput. Phys.* **2**(2), 255–270 (2007)
- PETSc Web page: <https://petsc.org/>.
- Ruge, J.: AMG for problems of elasticity. *Appl. Math. Comput.* **19**, 293–309 (1986)
- Ruge, J., Stüben, K.: Algebraic Multigrid, in *Multigrid Methods*, Frontiers Appl. Math. 3, S. McCormick (ed.), SIAM, Philadelphia, 73–130, 1987.
- Saad, Y.: Iterative methods for sparse linear systems, 2nd edn. SIAM, Philadelphia (2003)
- SAMG Web page: <https://www.scai.fraunhofer.de/en/business-research-areas/fast-solvers/products/samg.html>.
- Sellappan, V., Desam, B.: Accelerating ANSYS fluent simulations with NVIDIA GPUs. *ANSYS Advantage* **1X**(1), 51–53 (2015)

- Stüben, K.: Algebraic multigrid (AMG): an introduction with applications. In: Multigrid, U. Trottenberg, C. Oosterlee, A. Schuller, Academic Press, 413–532, (2001)
- Stüben, K., Delaney, P., Chmakov, S.: Algebraic multigrid for ground water flow and oil reservoir simulation. Proceedings of the Conference “MODFLOW and More 2003: Understanding through Modeling”, Intern. Ground Water Modeling Center (IGWMC) (2003)
- Stüben, K., Ruge, J., Clees, T., Gries, S., Plum, J.: Algebraic Multigrid - From Academia to Industry, in M. Griebel, A. Schuller, M. Schweitzer (eds.), Scientific Computing and Algorithms in Industrial Simulations, Springer, 2017.
- Tian, R., Zhou, M., Wang, J., Li, Y., An, H., Xu, X., Wen, L., Wang, L., Xu, Q., Leng, J., Xu, R., Zhang, B., Liu, W., Mo, Z.: A challenging dam structural analysis: large-scale implicit thermo-mechanical coupled contact simulation on Tianhe-II. *Comput. Mech.* **63**, 99–119 (2019)
- Trilinos Web page: <https://trilinos.github.io/>.
- Vanek, P., Mandel, J., Brezina, M.: Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems. *Computing* **56**, 179–196 (1996)
- Vanek, P., Brezina, M., Mandel, J.: Convergence of algebraic multigrid based on smoothed aggregation. *Numer. Math.* **88**, 559–579 (2001)
- Xiao, Y., Shu, S., Zhang, P., Mo, Z., Xu, J.: Semi-coarsening algebraic multigrid methods for 2D–3T energy equations. *J. Num. Method Comp. Appl.* **24**, 293–303 (2003)
- Xu, X.: Parallel algebraic multigrid methods: state-of-the art and challenges for extreme-scale applications. *J. Num. Method Comp. Appl.* **40**(4), 243–260 (2019)
- Xu, X., Mo, Z.: Algebraic interface based coarsening AMG preconditioner for multi-scale sparse matrices with applications to radiation hydrodynamics computation. *Numer. Linear Algebra Appl.* **24**(2), e2078 (2017)
- Xu, X., Mo, Z., An, H.: An algebraic two-level method for 2D–3T radiation diffusion equations. *Chinese J. Comput. Phys.* **26**(1), 1–8 (2009)
- Xu, X., Mo, Z., An, H.: An adaptive AMG preconditioning strategy for solving large-scale sparse linear systems. *Sci. Sin. Inf.* **46**(10), 1411–1420 (2016)
- Xu, X., Mo, Z., Yue, X., An, H., Shu, S.: α Setup-AMG: an adaptive-setup-based parallel AMG solver for sequence of sparse linear systems. *CCF Trans. HPC* **2**, 98–110 (2020)
- Yang, U.: On the use of relaxation parameters in hybrid smoothers. *Numer. Linear Algebra Appl.* **11**, 155–172 (2004)
- Yue, X., Shu, S., Xu, X., Zhou, Z.: An adaptive combined preconditioner with applications in radiation diffusion equations. *Commun. Comput. Phys.* **18**(5), 1313–1335 (2015)
- Yue, X., Xu, X., Shu, S.: JASMIN-based two-dimensional adaptive combined preconditioner for radiation diffusion equations in inertial fusion research. *East Asian J. Appl. Math.* **7**, 495–507 (2017)
- Yue, X., Zhang, S., Xu, X., Shu, S., Shi, W.: Algebraic multigrid block preconditioning for multi-group radiation diffusion equations. *Commun. Comput. Phys.* **29**(3), 831–852 (2021)
- Zhou, Z., Xu, X., Shu, S., Feng, C., Mo, Z.: Adaptive two-level preconditioners for 2D–3T radiation diffusion equations. *Chinese J. Comput. Phys.* **29**(4), 475–483 (2012)

Springer Nature or its licensor holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.